机械臂关节空间与笛卡尔空间轨迹规划

1.1 机械臂硬件连线与驱动准备

本实验使用基于 STM32 的多自由度总线舵机机械臂实机。在进行正逆解算法控制前，必须正确完成各关节舵机与控制板的物理连线。 如图 1.1-1 所示，底层控制板提供了 8 个舵机接口，根据实际机械臂拓扑结构，同图 1.1-1 中所示:

STM32 接口	接口 1	接口 2	接口 3	接口 4	接口 5	接口 6	接口 7	接口 8
机器人关节	预留	预留	关节 6	关节 2	关节 5	关节 4	关节 3	关节 1

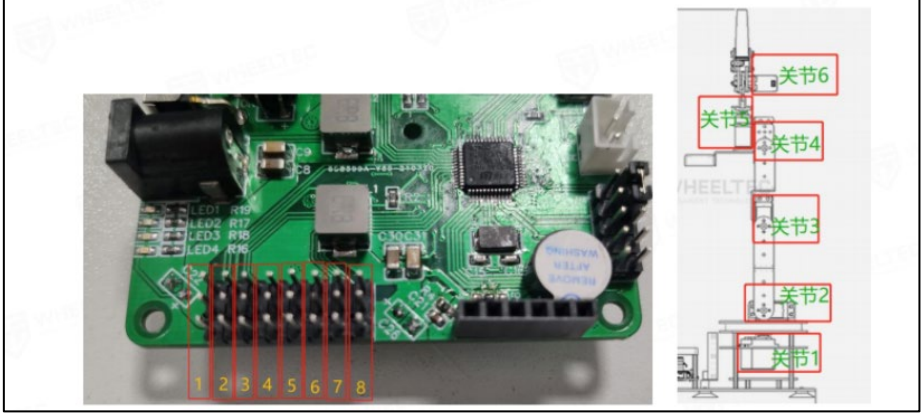


图 1.1-1

注意：硬件组装与接线时，请确保断电操作；接线完成后，再通过 USB 串口线将 STM32 控制板与计算机连接。USB 接口如图 1.1-2 所示:

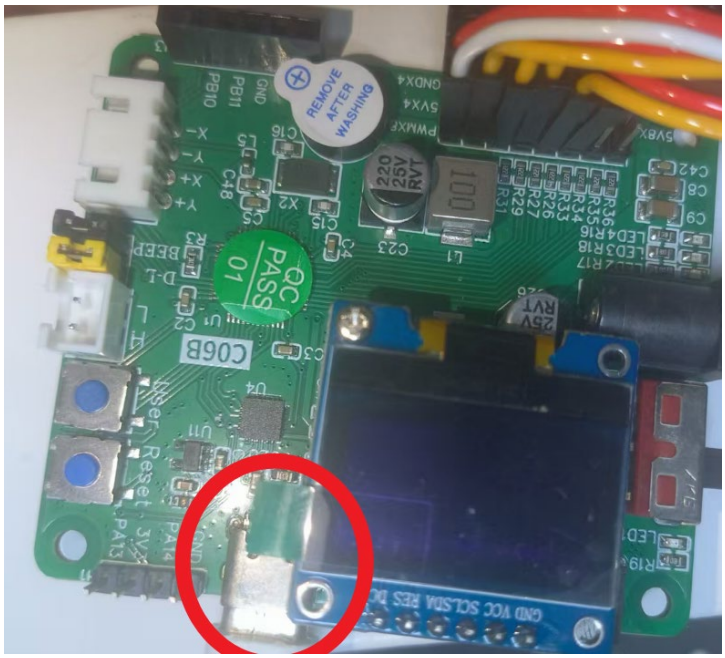


图 1.1-2

1.2 关节空间

1.2.1 实验目的

1. 理解关节空间轨迹规划（MoveJ）的基本原理与执行逻辑，理解各关节独立解耦插值、中间过渡轨迹不严格约束末端几何路径。
2. 了解起止端点位置、速度与加速度“双零边界条件”的物理意义，理解五次多项式消除机构加速度阶跃跳变、防止舵机齿轮冲击的理论基础。
3. 了解 Levenberg-Marquardt（LM）数值阻尼最小二乘逆解算法的调用方法，理解通过自由度掩码（Mask）实现末端空间位置解耦约束的技术细节。

1.2.2 实验环境

- **操作系统**：Windows 10/11
- **Python 运行环境**：Python 3.8 及以上版本
- **核心依赖包**：numpy、pyserial、roboticstoolbox-python、spatialmath-python、matplotlib
- **硬件设备**：五自由度总线舵机机械臂实验台、STM32 运动控制板、12V 稳压电源、通信数据线。

1.2.3 实验原理

在机器人工程实践中，若直接让机械臂从机械全零立正状态（各轴均为 $0°$）高速扑向远端工作点，不仅容易在奇异形位附近引发电机过载，还会导致末端轨迹不可控地甩动。

工业控制流程采用**多阶段时序调度**：

- **阶段 I**：先完成模型、逆解及整条轨迹规划，并播放仿真；若串口连接成功，再下发全零目标；
- **阶段 II**：从全零立正位姿 q_{zero} 平缓过渡到灵巧工作区内的“启动节点（准备姿态节点 1）” q_{start}，使各关节弯曲展开，避开坚直奇异形位；
- **阶段 III**：由逆解算法解算出末端目标点 Target A 对应的关节角度（节点 2）q_{goal}，规划从 q_{start} 到 q_{goal} 的光滑平稳轨迹并实时时序驱动；
- **阶段 IV**：作业完毕到位停靠后，再次平缓受控降速回退至零位待机。

在关节空间中，各关节独立进行标量插值。设某一关节从起始时刻 $t = 0$（角度为 q_0）运动至终止时刻 $t = T$（角度为 q_f）。为了保证机构在启动和停靠瞬间不发生机械冲击，对机构施加**位置、速度、加速度双零边界条件**：

$$\begin{cases} q(0) = q_0, & q(T) = q_f \\ \dot{q}(0) = 0, & \dot{q}(T) = 0 \\ \ddot{q}(0) = 0, & \ddot{q}(T) = 0 \end{cases}$$

构造五次位移多项式方程：

$$q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5$$

对时间求导分别得到速度 $\dot{q}(t)$ 与加速度 $\ddot{q}(t)$：

$$\dot{q}(t) = a_1 + 2a_2 t + 3a_3 t^2 + 4a_4 t^3 + 5a_5 t^4$$

$$\ddot{q}(t) = 2a_2 + 6a_3 t + 12a_4 t^2 + 20a_5 t^3$$

将 $t = 0$ 处的初值条件代入，直接求得：

$$a_0 = q_0, \quad a_1 = 0, \quad a_2 = 0$$

再将 $t = T$ 处的终值条件代入方程组求解，可得到归一化时间参数 $\tau = \frac{t}{T} \in [0,1]$ 下的标量插值多项式 $s(\tau)$：

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5$$

由此得到任意离散时刻 t_k 的关节位置递推方程：

$$q(t_k) = q_0 + s\left(\frac{t_k}{T}\right) \cdot (q_f - q_0)$$

由于 $s(\tau)$ 的一阶导数 $\dot{s}(0) = \dot{s}(1) = 0$、二阶导数 $\ddot{s}(0) = \ddot{s}(1) = 0$，该插值算法保证了关节角速度与角加速度在整段运动区间内完全平滑连续，彻底消除了阶跃过载。

机械臂实机共有 5 个驱动自由度（底座偏航、大臂俯仰、小臂俯仰、腕部俯仰、手爪旋转），其末端无法在三维空间中同时任意指定 6 个维度的独立位姿（3 个位置自由度 + 3 个姿态自由度）。

针对末端到达指定空间坐标 $P_A = [X_A, Y_A, Z_A]^T$ 的作业需求，采用 Levenberg-Marquardt（LM）带阻尼数值优化算法求解逆解，并引入自由度掩码矩阵：

$$\text{mask} = [1,\ 1,\ 1,\ 0,\ 0,\ 0]$$

该掩码指示求解器仅对末端的空间平移误差 $[e_x, e_y, e_z]^T$ 进行收敛约束，而将末端的姿态自由度予以放开，从而确保欠自由度机械臂能稳定、快速地求得目标位姿的逆运动学解。

Rtb 函数功能介绍:

本实验主要调用了 roboticstoolbox-python（RTB）与 spatialmath-python（SM）库中的核心函数，其作用与接口说明如下：

- **spatialmath.SE3(x, y, z)**：构建特殊欧式群 $SE(3)$ 齐次变换矩阵对象。用于表示目标点在笛卡尔三维空间中的绝对平移坐标，在未指定旋转矩阵时默认姿态为单位阵。
- **rtb.RevoluteDH(d, a, alpha, offset)**：构建标准 Denavit-Hartenberg（D-H）旋转关节连杆对象。四个参数分别代表连杆偏置 d、连杆长度 a、连杆扭转角 α 以及关节初始角度偏置 offset。
- **rtb.DHRobot(links, name)**：将一组定义好的 D-H 连杆对象串联装配成机器人运动学模型对象，提供正解、逆解及三维绘图等方法。
- **robot.ikine_LM(T, q0, mask)**：采用 Levenberg-Marquardt 阻尼最小二乘法进行数值逆运动学求解。T 为目标 $SE(3)$ 位姿，q0 为求解迭代的初始种子姿态，mask 为自由度加权掩码。函数返回一个解对象，通过 sol.success 判断求解是否收敛，通过 sol.q 提取解出

的关节角度向量。

- **rtb.jtraj(q0, qf, m)**：五次多项式关节空间轨迹规划器。传入起始关节向量 q_0、目标关节向量 q_f 以及离散采样步数 m，内部自动构造双零边界条件多项式进行插值，返回包含位置 .q、速度 .qd 和加速度 .qdd 的轨迹序列对象。
- **robot.plot(q, limits, ...)**：基于 Matplotlib 绘制机械臂三维骨架模型，并可传入轨迹矩阵逐帧回放动画，用于离线验证规划轨迹的正确性。

1.2.4 实验要求

任务目标：学生需独立设计并补全主控调度函数 run_movej_experiment()，驱动实体机械臂实现以下运动流程：

- **启动准备**：从全零立正位姿 $q_{zero} = [0°, 0°, 0°, 0°, 0°]$ 出发，平稳运行至启动节点（准备姿态节点 1）$q_{start} = [0°, 0°, 20°, 10°, 0°]$；
- **逆解作业**：在上位机中解算宽裕目标点 Target A（$P_A = [120.0,80.0,220.0]$ mm）的关节逆解 q_{goal}，规划从节点 1 到节点 2 的平缓轨；
- **安全回零**：在目标点平稳停靠 1.5 秒后，将机械臂安全、缓慢地复位至全零待命位姿，并关闭串口。

1.2.5 实验步骤

步骤 1: 硬件通信初始化与上电立正。调用 init_serial() 函数打开控制板串口，下发全零帧复位指令，机械臂各轴进入全零立正位姿 $q_{zero} = [0°, 0°, 0°, 0°, 0°]$，建立上位机与下位机的数据链路。

步骤 2: 机器人物理 D-H 模型装配。基于实机物理尺寸加载 D-H 参数表（包含底座回转、大臂、小臂、腕俯仰、腕横滚轴），调用 rtb.RevoluteDH 与 rtb.DHRobot 装配得到机器人运动学模型对象。

步骤 3: 目标点逆运动学求解与安全性校验。定义宽裕作业目标点 A（$P_A = [120.0,80.0,220.0]$ mm），调用 sm.SE3 构建目标位姿矩阵；配置初始迭代种子角度（设定肘部向上预弯曲构型）与位置掩码 mask=[1, 1, 1, 0, 0, 0]，调用 robot.ikine_LM() 求解目标关节角 q_{goal}；若逆解失败（ik_sol.success == False）则终止下发，避免危险动作。

步骤 4: 阶段一轨迹规划（全零位至启动节点）。设定启动节点（准备姿态）$q_{start} = [0°, 0°, 20°, 10°, 0°]$，调用 rtb.jtraj() 生成关节空间平滑轨迹 traj1；先与 traj2、traj_ret 一起仿真，再在串口连接成功后调用 execute_joint_trajectory(ser, traj1.q, duration=2.0) 驱动机械

臂展开。

步骤 5: 阶段二主作业轨迹规划与虚拟动画预演。调用 rtb.jtraj() 生成从启动节点 q_{start} 至目标逆解角度 q_{goal} 的五次多项式轨迹 traj2；连同回零段先调用 animate_robot_motion() 播放三维动画和末端轨迹，再尝试连接 COM3。

步骤 6: 阶段二实机驱动与到位停靠。调用硬件执行函数 execute_joint_trajectory(ser, traj2.q, duration=3.0)，驱动实体机械臂运行至 Target A 目标点，并通过 time.sleep(1.5) 保持静止停靠以消除机械残余抖动。

步骤 7: 阶段三安全受控回零与通信释放。调用 rtb.jtraj() 生成从当前目标点 q_{goal} 退回全零位姿 q_{zero} 的复位轨迹 traj_ret；调用 execute_joint_trajectory(ser, traj_ret.q, duration=3.0) 驱动实机平稳归位，最后关闭串口通信链路并释放图形资源。

1.2.6 **示例代码**

任务代码:

```
import argparse
import time
import serial
import numpy as np
import spatialmath as sm
import roboticstoolbox as rtb
import matplotlib

try:
    matplotlib.use('TkAgg')
except Exception:
    pass
import matplotlib.pyplot as plt

# 模型角先变控制侧角，再转换为舵机角。
# 两次 -1 的净效果是发送角度与模型角度一致。
SERVO_DIRECTION_MAP = np.full(6, -1.0)
```

```
def pack_frame(q_rad):
    """
    函数作用:
        将 6 轴关节弧度数组打包构建为符合 STM32 底层控制协议规范的 16 字节十六进制通信帧。

    参数说明:
        q_rad (np.ndarray 或 list):
            长度为 6 的浮点型数组，表示各关节电机的目标位置，单位为弧度 (rad)。
            数据顺序依次映射为:
            [关节 1(基座偏航)，关节 2(肩部立柱)，关节 3(大臂俯仰)，关节 4(小臂俯仰)，关节 5(腕部横滚)，关节 6(夹爪)]

    返回值:
        bytearray: 长度为 16 的无符号字节数组，直接用于串口二进制发送。

    协议与输入限制:
        1. 帧结构:
           - Byte 0: 固定帧头 (0xAA)
           - Byte 1~12: 6 个关节的有符号 16 位整数数据 (Big-Endian 大端模式，高 8 位在前，低 8 位在后)
           - Byte 13: 控制模式字 (0x01 表示舵机绝对位置控制)
           - Byte 14: 前 14 字节的逐字节按位异或校验和 (XOR Checksum)
           - Byte 15: 固定帧尾 (0xBB)
        2. 数据定点化: 实际计算公式为 int(q_rad[i] * 1000)，分辨率为 0.001 rad (~0.057°)。
        3. 幅值安全限幅: 各关节计算值被钳位在 [-32768, 32767] 内，防止整形数值溢出。
    """
    tx = bytearray(16)
    tx[0] = 0xAA  # 帧头
    for i in range(6):
        val = int(q_rad[i] * 1000)
        val = max(min(val, 32767), -32768)
        tx[1 + i * 2] = (val >> 8) & 0xFF
        tx[2 + i * 2] = val & 0xFF
    tx[13] = 0x01  # 控制模式标识

    check = 0
    for b in tx[:14]:
        check ^= b
    tx[14] = check
    tx[15] = 0xBB  # 帧尾
```

```
    return tx



def init_serial(port="COM3", baudrate=115200):
    """
    函数作用:
        安全初始化主机与 STM32 控制板的串口通信链路，并在连接成功后自动执行上电安全回零。

    参数说明:
        port (str):
            串口设备通信标识符。在 Windows 下通常为 'COM3', 'COM5' 等；在 Linux 下通常为 '/dev/ttyUSB0'。
        baudrate (int):
            串口通信波特率，默认为 115200 bps。

    返回值:
        serial.Serial 或 None:
            若端口成功打开则返回串口对象实例；若串口打开异常或未检测到设备，返回 None 。
    """
    ser = serial.Serial()
    ser.port = port
    ser.baudrate = baudrate
    ser.timeout = 1
    ser.dtr = False
    ser.rts = False
    try:
        ser.open()
    except Exception as e:
        print(f"未检测到物理串口 {port} ({e})；仅运行仿真")
        return None
    try:
        print(f"成功建立控制板串口通信: {port}")
        time.sleep(1.0)
        ser.write(pack_frame(np.zeros(6, dtype=np.float64)))
        ser.flush()
        time.sleep(1.5)
        print("已发送六轴全零目标；串口协议不提供实际到位反馈。")
        return ser
    except Exception:
```

```
        ser.close()
        raise



def build_robot_model(dh_params, name="Physical_Servo_Arm"):
    """
        函数作用:
            接收标准 D-H 参数列表，利用 RoboticsToolbox 构造机械臂的虚拟运动学模型对象。

        参数说明:
            dh_params (list of dict):
                包含各连杆标准 D-H 四参数及校准偏量的字典列表。每个字典元素必须包含如下键名:
                - 'd': 沿上一转轴的偏距 (float, 单位: mm)
                - 'a': 沿当前公垂线的长度 (float, 单位: mm)
                - 'alpha': 绕公垂线的连杆扭角 (float, 单位: rad)
                - 'offset': 初始零位校准偏量 (float, 单位: rad, 可选，默认为 0.0)
            name (str):
                分配给机器人模型的命名标识字符串。

        返回值:
            rtb.DHRobot:
                配置完毕的标准 DHRobot 运动学模型对象，可直接调用 .fkine() 及 .plot() 接口。
        """
    links = [
        rtb.RevoluteDH(
            d=p['d'],
            a=p['a'],
            alpha=p['alpha'],
            offset=p.get('offset', 0.0)
        )
        for p in dh_params
    ]
    return rtb.DHRobot(links, name=name)



def animate_robot_motion(robot, q_trajectory, dt=0.04, target_point=None):
    """
        函数作用:
```

```
        在三维图形视窗中加载机械臂几何模型，并按指定帧率动态播放输入的连续关节轨迹动画。

    参数说明:
        robot (rtb.DHRobot):
            由 build_robot_model() 装配生成的机器人运动学模型对象。
        q_trajectory (np.ndarray):
            形状为 (N, 5) 的二维浮点型数组，表示已规划完成的 N 个离散时刻的 5 轴关节角度序列 (单位: rad)。
        dt (float):
        动画每帧渲染的刷新时间间隔，单位为秒 (s)，默认为 0.04 秒 (约 25 FPS)。
    target_point (array-like 或 None):
        可选的作业目标点，用于在末端轨迹上标出 Target A。

    返回值:
        None (无返回值，直接更新屏幕三维视窗并驻留 1.0 秒供观察)。
    """
# 先用整条关节轨迹正解计算末端位置；灰色虚线是规划路径，
# 橙色实线随着机械臂播放逐帧延长，包含回零过程。
tool_path = np.asarray([robot.fkine(q).t for q in q_trajectory])
env = robot.plot(
    q_trajectory[0],
    block=False,
    limits=[-150, 450, -250, 250, 0, 500],
    jointaxes=True,
    shadow=True,
    dt=dt,
)
ax = env.ax
ax.plot(*tool_path.T, '--', color='0.55', linewidth=1.4, label='Planned TCP path')
if target_point is not None:
    ax.scatter(*target_point, color='crimson', s=35, label='Target A')
traced, = ax.plot([], [], [], color='darkorange', linewidth=2.5, label='TCP trace')
ax.legend(loc='upper left')
ax.set_title('MoveJ - end-effector path')
for k, q in enumerate(q_trajectory):
    robot.q = q
    traced.set_data(tool_path[:k + 1, 0], tool_path[:k + 1, 1])
    traced.set_3d_properties(tool_path[:k + 1, 2])
    env.step(dt)
```

```
    plt.pause(1.0)



def execute_joint_trajectory(ser, q_traj_rad, duration=3.0):
    """
        函数作用:
            接收由轨迹规划算法计算生成的连续关节空间位置矩阵 (N x 5)，自动将其维度对齐补全
            为符合 STM32 控制板硬件协议规范的 6 轴控制数据，并以严格的时间节拍流式下发至实机执行。

        参数说明:
            ser (serial.Serial 或 None):
                由 init_serial() 返回的串口通信对象实例。
                - 若为有效串口对象，则执行物理硬件写入;
                - 若为 None 或串口未开启，系统自动跳过硬件下发，保障虚拟仿真模式平稳运行。
            q_traj_rad (np.ndarray):
                形状为 (N, 5) 的二维双精度浮点型数组，表示规划生成的 N 个时间离散时刻的 5 轴关节角度序列，
                单位为弧度 (rad)。
            duration (float):
                整段轨迹在实体机械臂上执行的总规划时长，单位为秒 (s)，默认为 3.0 秒。

        返回值:
            None (无返回值)。

        """
    if ser is None or not ser.is_open:
        print("无物理硬件连接，跳过硬件时序驱动。")
        return

    steps = len(q_traj_rad)
    dt = duration / steps
    for k in range(steps):
        q_model = np.append(q_traj_rad[k], 0.0)  # 第 6 轴夹爪保持零度
        q_control = q_model * SERVO_DIRECTION_MAP
        q_servo = q_control * SERVO_DIRECTION_MAP
        ser.write(pack_frame(q_servo))
        ser.flush()
        time.sleep(dt)
```

```
def run_movej_experiment(show_animation=True, mode="auto", port="COM3"):
    """任务：补全轨迹规划、仿真以及按模式执行的主控流程。"""
    pass



if __name__ == '__main__':
    parser = argparse.ArgumentParser(description="第七周实验一：关节空间 MoveJ")
    parser.add_argument("--mode", choices=("auto", "simulate", "hardware"), default="auto")
    parser.add_argument("--port", default="COM3", help="auto/hardware 模式使用，默认 COM3")
    parser.add_argument("--no-animation", action="store_true")
    args = parser.parse_args()
    run_movej_experiment(show_animation=not args.no_animation, mode=args.mode, port=args.port)
```

完整示例代码:

```
# 请根据上方任务代码独立补全 MoveJ 实验，此处不提供完整实现。
```

1.3 笛卡尔空间

1.3.1 实验目的

理解笛卡尔空间规划与关节空间规划的物理本质差异：深入对比关节空间规划（MoveJ）与笛卡尔空间规划（MoveL / MoveC）在数学描述空间、末端几何路径约束以及活动关节速度协同性上的本质区别。

掌握笛卡尔直线与空间圆弧的几何插补原理：掌握模型末端沿空间直线插值（MoveL）与空间三点外接圆平面插补（MoveC）的解析几何构建方法。

理解逐点在线逆解映射机制及其物理代价：掌握在笛卡尔连续轨迹上逐点调用数值逆解（ikine_LM）的技术，观察并理解欠自由度机械臂在无复杂后处理优化算法干预时，由于冗余解空间特性导致关节运动呈现非线性与伴随性微振颤的物理机理。

完成笛卡尔轨迹的时序调度与实机驱动验证：掌握从全零位平滑展开至轨迹起点、执行笛卡尔几何轨迹跟踪、到位平稳停靠及安全回零的完整控制流程。

1.3.2 实验环境

- **操作系统**：Windows 10 / 11
- **Python 运行环境**：Python 3.8 及以上版本
- **核心依赖库**：numpy、pyserial、roboticstoolbox-python、spatialmath-python、matplotlib
- **硬件设备**：基于 STM32 的五自由度总线舵机机械臂实验台、12V 稳压电源、USB 通信数据线。

1.3.3 实验原理

关节空间与笛卡尔空间

关节空间规划（MoveJ）：直接在关节角空间对起止构型进行多项式插值。各关节电机单调平滑转动，加速度无阶跃跳变，插补过程中**不保证避开运动学奇异点**。其物理代价是末端在空间中画出一条随机构弯曲的自然空间弧线，无法精确约束末端的几何行进路线。

笛卡尔空间规划（MoveL / MoveC）：先规划模型末端在三维几何空间中的目标曲线（如直线或圆弧）。该模式常用于焊接走缝、激光切割及胶水喷涂等严苛工艺。其物理代价是各关节电机必须做出高度复杂的非线性联动；由于该机械臂为 5 自由度构型，仅约束空间位置时存在冗余自由度，导致底层数值逆解在各离散步之间极易产生微小波动，

在没有施加复杂平滑滤波优化算法时，机构在运行过程中会呈现出特有的微振颤与非线性折屈动作。

笛卡尔空间直线插补原理（MoveL）

设笛卡尔空间直线的起点坐标为 $P_{start} = [X_0, Y_0, Z_0]^T$，终点坐标为 $P_{goal} = [X_f, Y_f, Z_f]^T$。总规划时间为 T，插补离散步数为 N，单步时间步长为 $\Delta t = \frac{T}{N}$。

引入归一化时间标量 $\tau = \frac{t_k}{T} \in [0,1]$（其中 $t_k = k \cdot \Delta t$），任意离散时刻末端在笛卡尔三维空间中的坐标递推方程为：

$$P(t_k) = P_{start} + s(\tau) \cdot (P_{goal} - P_{start})$$

其中 $s(\tau)$ 可采用线性插值或多项式插值，生成的离散点在空间中严格满足三维直线几何方程。

空间三点圆弧插补原理（MoveC）

空间中任意不共线的三个笛卡尔坐标点 P_{start}（起点）、P_{via}（中间过渡点）、P_{goal}（终点）唯一确定一个外接圆平面与圆周路径：

1. **法向量与圆心解析推导**：通过向量外积构造圆平面的法向量 $\vec{n} = \frac{(P_{via}-P_{start})\times(P_{goal}-P_{start})}{\|(P_{via}-P_{start})\times(P_{goal}-P_{start})\|}$；基于外心重心坐标公式解析求解外接圆圆心坐标 P_c 与外接圆半径 R。
2. **正交基底投影与跨界判定**：以圆心 P_c 为原点构建圆平面的二维正交基底 $(\vec{u}, \vec{v})$，将中间点与终点投影并计算其有向极角 θ_1 与 θ_2。通过判断 θ_2 与 θ_1 的几何先后次序，确定总旋转跨角 θ_{total}，确保运动轨迹严格经由中间点到达终点。
3. **圆周时间参数化**：将归一化时间多项式施加于极角 $\theta(t_k) = s(\tau) \cdot \theta_{total}$，并通过基底逆变换恢复三维空间坐标：

$$P(t_k) = P_c + R\cos(\theta(t_k)) \cdot \vec{u} + R\sin(\theta(t_k)) \cdot \vec{v})$$

逐点在线数值逆解映射与冗余轴锁定

生成笛卡尔几何点列后，必须逐点映射为各关节电机的驱动指令：

- **自由度掩码**：机械臂臂体为 5 自由度，末端无法同时约束 6 自由度位姿，调用求解器时需传入位置掩码 mask = [1,1,1,0,0,0]，放开末端姿态自由度；
- **前向初值链递推**：求解第 k 步逆解时，将第 $k-1$ 步计算所得的关节角度作为初始猜测初值种子（$q_{seed} = q(t_{k-1})$），引导数值迭代在相邻解分支中平滑过渡；
- **腕部横滚锁零**：由于第 5 轴对末端空间位置无几何贡献，解算后强制将其置零（q_curr[4] = 0.0），消除冗余轴的随机漂移。

核心算法工具箱（RoboticsToolbox & SpatialMath）关键函数介绍

- **spatialmath.SE3(x, y, z)**：构建特殊欧式群 $SE(3)$ 齐次变换矩阵对象，封装空间三维坐标点。
- **rtb.ctraj(T0, T1, steps)**：笛卡尔空间连续位姿轨迹插补器，生成起始与目标位姿之

间的连续 $SE(3)$ 序列。

- **robot.ikine_LM(T, q0, mask)**：阻尼最小二乘数值逆解求解器，传入目标位姿 T、初值种子 q0 与位置掩码 mask，逐点计算关节向量。
- **rtb.jtraj(q0, qf, steps)**：关节空间五次多项式轨迹规划器，用于全零位平滑展开与安全回零。

1.3.4 实验要求

任务目标：编写并完善主控函数 run_movel_experiment()，驱动实体机械臂实现从全零立正位姿展开、追踪 MoveL 直线到达 Target A 或 MoveC 水平圆弧到达其独立终点，并平稳回零复位的完整时序控制流程。

规划方式选择：**示例同时实现直线插补（MoveL）与圆弧插补（MoveC）**（默认执行 MoveL；运行时用 --motion movec 选择圆弧，或在 --mode simulate 下用 --motion both 连续预览），完成笛卡尔轨迹到物理实机的驱动验证。 **本实验不包含插值的优化算法**，可以忽略优化过程，对比笛卡尔空间与关节空间即可。

空间点位参数：

- 直线/圆弧起点（笛卡尔坐标）：$P_{start} = [120.0,0.0,260.0]$ mm
- 圆弧中间过渡点（笛卡尔坐标，仅 MoveC）：$P_{via} = [160.0,40.0,260.0]$ mm
- MoveL 终点（笛卡尔坐标 Target A）：$P_{goal} = [120.0,80.0,220.0]$ mm；MoveC 终点：(120.0,80.0,260.0) mm。

1.3.5 实验步骤

步骤 1: 确认运行模式。默认 auto 模式先规划整条路径并仿真，随后才尝试 COM3；simulate 模式不打开串口。连接成功后，init_serial() 下发全零目标，对应模型位姿 $q_{zero} = [0°, 0°, 0°, 0°, 0°]$；串口协议不读取实机到位反馈。

步骤 2: 机器人物理 D-H 模型装配。基于实机物理尺寸加载 D-H 参数表（包含底座回转、大臂、小臂、腕俯仰、腕横滚轴），调用 rtb.RevoluteDH 与 rtb.DHRobot 装配得到机器人运动学模型对象。

步骤 3: 笛卡尔几何端点设定。定义空间起点 $P_{start} = [120.0,0.0,260.0]$ mm 与 MoveL 终点 Target A（$P_{goal} = [120.0,80.0,220.0]$ mm）；若选择 MoveC，同时定义等高的圆弧过渡点 $P_{via} = [160.0,40.0,260.0]$ mm，并设 MoveC 终点为 (120.0,80.0,260.0) mm。

步骤 4: 笛卡尔几何路径生成（任选其一执行）：

- **选项 A（空间直线 MoveL）**：调用 sm.SE3 分别封装起点与终点，调用

rtb.ctraj(T_start, T_goal, steps) 生成空间连续直线点列；

- **选项 B（空间圆弧 MoveC）**：调用预提供的 plan_cartesian_arc(p_start, p_via, p_goal, steps) 生成空间三点外接圆几何点列，并封装为连续位姿序列。

步骤 5: 逐点在线数值逆运动学求解。配置初始迭代种子 q_seed = [0°, 45°, −80°, −80°, 0°] 与位置掩码 mask=[1, 1, 1, 0, 0, 0]；遍历所选路径的几何点列，循环调用 robot.ikine_LM() 求解各点关节角，将当前解作为下一时刻的初值种子，并将第 5 轴角度固定为零，获得关节轨迹矩阵 q_traj。

步骤 6:提取轨迹起点对应的关节构型 q_traj[0]，调用 rtb.jtraj(q_zero, q_traj[0], 40) 计算展开轨迹；仿真整条轨迹后，若 COM3 连接成功，再由 execute_joint_trajectory() 驱动实机展开至起点。

步骤 7: 虚拟动画预演与实机作业。animate_robot_motion() 先显示三维模型、规划路径和末端已走轨迹；然后尝试连接 COM3。连接成功后，调用 execute_joint_trajectory(ser, q_traj, duration=4.0) 驱动实体机械臂追踪所选路径至相应终点，并在到位后保持静止 1.5 秒。

步骤 8: 调用 rtb.jtraj() 生成从所选路径终点平缓返回全零立正位姿 q_{zero} 的复位轨迹；调用 execute_joint_trajectory() 驱动实机平稳归位，最后关闭串口通信链路并释放图形资源。

1.3.6 示例代码

任务代码:

```
import argparse
import time
import serial
import numpy as np
import spatialmath as sm
import roboticstoolbox as rtb
import matplotlib

try:
    matplotlib.use('TkAgg')
except Exception:
    pass
import matplotlib.pyplot as plt

# 模型角到控制侧角、控制侧角到舵机角各乘一次 -1。
SERVO_DIRECTION_MAP = np.full(6, -1.0)
```

```
def pack_frame(q_rad):
    """
    函数作用:
        将 6 轴关节弧度数组打包构建为符合 STM32 底层控制协议规范的 16 字节十六进制通信帧。

    参数说明:
        q_rad (np.ndarray 或 list):
            长度为 6 的浮点型数组，表示各关节电机的目标位置，单位为弧度 (rad)。
            数据顺序依次映射为:
            [关节 1(基座偏航)，关节 2(肩部立柱)，关节 3(大臂俯仰)，关节 4(小臂俯仰)，关节 5(腕部横滚)，关节 6(夹爪)]

    返回值:
        bytearray: 长度为 16 的无符号字节数组，直接用于串口二进制发送。

    协议与输入限制:
        1. 帧结构:
           - Byte 0: 固定帧头 (0xAA)
           - Byte 1~12: 6 个关节的有符号 16 位整数数据 (Big-Endian 大端模式，高 8 位在前，低 8 位在后)
           - Byte 13: 控制模式字 (0x01 表示舵机绝对位置控制)
           - Byte 14: 前 14 字节的逐字节按位异或校验和 (XOR Checksum)
           - Byte 15: 固定帧尾 (0xBB)
        2. 数据定点化: 实际计算公式为 int(q_rad[i] * 1000)，分辨率为 0.001 rad (~0.057°)。
        3. 幅值安全限幅: 各关节计算值被钳位在 [-32768, 32767] 内，防止整形数值溢出。
    """
    tx = bytearray(16)
    tx[0] = 0xAA  # 帧头
    for i in range(6):
        val = int(q_rad[i] * 1000)
        val = max(min(val, 32767), -32768)  # 硬件限幅
        tx[1 + i * 2] = (val >> 8) & 0xFF
        tx[2 + i * 2] = val & 0xFF
    tx[13] = 0x01  # 控制模式标识

    check = 0
    for b in tx[:14]:
        check ^= b
```

```
    tx[14] = check
    tx[15] = 0xBB  # 帧尾
    return tx



def init_serial(port="COM3", baudrate=115200):
    """
    函数作用:
        安全初始化主机与 STM32 控制板的串口通信链路，并在连接成功后自动执行上电安全回零。

    参数说明:
        port (str):
            串口设备通信标识符。在 Windows 下通常为 'COM3', 'COM5' 等；在 Linux 下通常为 '/dev/ttyUSB0'。
        baudrate (int):
            串口通信波特率，默认为 115200 bps。

    返回值:
        serial.Serial 或 None:
            若端口成功打开则返回串口对象实例；若串口打开异常或未检测到设备，返回 None 。
    """
    ser = serial.Serial()
    ser.port = port
    ser.baudrate = baudrate
    ser.timeout = 1
    ser.dtr = False
    ser.rts = False
    try:
        ser.open()
    except Exception as e:
        print(f"未检测到物理串口 {port} ({e})；仅运行仿真")
        return None
    try:
        print(f"成功建立控制板串口通信: {port}")
        time.sleep(1.0)
        ser.write(pack_frame(np.zeros(6, dtype=np.float64)))
        ser.flush()
        time.sleep(1.5)
        print("已发送六轴全零目标；串口协议不提供实际到位反馈。")
```

```
        return ser
    except Exception:
        ser.close()
        raise



def build_robot_model(dh_params, name="Physical_Servo_Arm"):
    """
    函数作用:
        接收标准 D-H 参数列表，利用 RoboticsToolbox 构造机械臂的虚拟运动学模型对象。

    参数说明:
        dh_params (list of dict):
            包含各连杆标准 D-H 四参数及校准偏量的字典列表。每个字典元素必须包含如下键名:
            - 'd': 沿上一转轴的偏距 (float, 单位: mm)
            - 'a': 沿当前公垂线的长度 (float, 单位: mm)
            - 'alpha': 绕公垂线的连杆扭角 (float, 单位: rad)
            - 'offset': 初始零位校准偏量 (float, 单位: rad, 可选，默认为 0.0)
        name (str):
            分配给机器人模型的命名标识字符串。

    返回值:
        rtb.DHRobot:
            配置完毕的标准 DHRobot 运动学模型对象，可直接调用 .fkine() 及 .plot() 接口。
    """
    links = [
        rtb.RevoluteDH(
            d=p['d'],
            a=p['a'],
            alpha=p['alpha'],
            offset=p.get('offset', 0.0)
        )
        for p in dh_params
    ]
    return rtb.DHRobot(links, name=name)



def animate_robot_motion(robot, q_trajectory, dt=0.04, reference_points=None,
```

```
                    via_point=None, motion="movel", path_start_index=0):
"""
函数作用:
    在三维图形视窗中加载机械臂几何模型，并按指定帧率动态播放输入的连续关节轨迹动画。

参数说明:
    robot (rtb.DHRobot):
        由 build_robot_model() 装配生成的机器人运动学模型对象。
    q_trajectory (np.ndarray):
        形状为 (N, 5) 的二维浮点型数组，表示已规划完成的 N 个离散时刻的 5 轴关节角度序列 (单位: rad)。
    dt (float):
        动画每帧渲染的刷新时间间隔，单位为秒 (s)，默认为 0.04 秒 (约 25 FPS)。
    reference_points (array-like 或 None):
        MoveL/MoveC 的笛卡尔规划点列，绘为蓝色路径。
    via_point (array-like 或 None):
        MoveC 的三点定圆过渡点。
    path_start_index (int):
        笛卡尔作业段在完整关节轨迹中的首帧下标，用于放大视图同步更新。

返回值:
    None (无返回值，直接更新屏幕三维视窗并驻留 1.0 秒供观察)。
"""
# 完整进场、笛卡尔运动、回零的末端正解曲线与目标路径分别显示。
tool_path = np.asarray([robot.fkine(q).t for q in q_trajectory])
env = robot.plot(
    q_trajectory[0],
    block=False,
    limits=[-150, 450, -250, 250, 0, 500],
    jointaxes=True,
    shadow=True,
    dt=dt,
)
ax = env.ax
env.timer.set_visible(False)  # 避免仿真计时文字压住右上角路径放大视图。
ax.plot(*tool_path.T, '--', color='0.65', linewidth=1.2, label='Planned TCP path')
if reference_points is not None:
    reference_points = np.asarray(reference_points)
    # 目标路径与橙色实走轨迹会重合；用置顶蓝色虚线保留二者的可见性。
```

```
    ax.plot(*reference_points.T, '--', color='royalblue', linewidth=2.8,
            zorder=15, label=f'{motion.upper()} reference')
    ax.scatter(*reference_points[0], color='green', s=35, label='Path start')
    ax.scatter(*reference_points[-1], color='crimson', s=35, label='Path goal')
if via_point is not None:
    ax.scatter(*via_point, color='purple', s=35, label='Arc via point')
traced, = ax.plot([], [], [], color='darkorange', linewidth=2.2, label='TCP trace')
ax.legend(loc='upper left')
ax.set_title(f'{motion.upper()} - end-effector path')

# 全机械臂视图包含回零高度，作业段在其中较小；右上角显示作业平面。
detail_trace = None
if reference_points is not None:
    plane_axes = (0, 1) if motion == "movec" else (1, 2)
    horizontal_axis, vertical_axis = plane_axes
    plane_name = 'X-Y' if motion == "movec" else 'Y-Z'
    detail = env.fig.add_axes([0.64, 0.69, 0.30, 0.26])
    detail.plot(reference_points[:, horizontal_axis],
                reference_points[:, vertical_axis], '--',
                color='royalblue', linewidth=3.2)
    detail.scatter(reference_points[0, horizontal_axis],
                   reference_points[0, vertical_axis],
                   color='green', s=25, zorder=5)
    detail.scatter(reference_points[-1, horizontal_axis],
                   reference_points[-1, vertical_axis],
                   color='crimson', s=25, zorder=5)
    if via_point is not None:
        detail.scatter(via_point[horizontal_axis], via_point[vertical_axis],
                       color='purple', s=25, zorder=5)
    detail_trace, = detail.plot([], [], color='darkorange', linewidth=1.5)
    low = reference_points[:, plane_axes].min(axis=0) - 10.0
    high = reference_points[:, plane_axes].max(axis=0) + 10.0
    detail.set_xlim(low[0], high[0])
    detail.set_ylim(low[1], high[1])
    detail.set_xlabel(f'{plane_name[0]} (mm)', fontsize=8)
    detail.set_ylabel(f'{plane_name[-1]} (mm)', fontsize=8)
    detail.set_title(f'{motion.upper()} TCP path ({plane_name})', fontsize=9)
    detail.tick_params(labelsize=7)
```

```
        detail.grid(True, alpha=0.3)


    for k, q in enumerate(q_trajectory):
        robot.q = q
        traced.set_data(tool_path[:k + 1, 0], tool_path[:k + 1, 1])
        traced.set_3d_properties(tool_path[:k + 1, 2])
        if detail_trace is not None and k >= path_start_index:
            path_end = min(k + 1, path_start_index + len(reference_points))
            detail_trace.set_data(
                tool_path[path_start_index:path_end, horizontal_axis],
                tool_path[path_start_index:path_end, vertical_axis])
        env.step(dt)
    plt.pause(1.0)



def execute_joint_trajectory(ser, q_traj_rad, duration=3.0):
    """
    函数作用:
        接收由轨迹规划算法计算生成的连续关节空间位置矩阵 (N x 5)，自动将其维度对齐补全
        为符合 STM32 控制板硬件协议规范的 6 轴控制数据，并以时间节拍流式下发至实机执行。

    参数说明:
        ser (serial.Serial 或 None):
            由 init_serial() 返回的串口通信对象实例。
            - 若为有效串口对象，则执行物理硬件写入;
            - 若为 None 或串口未开启，系统自动跳过硬件下发，保障虚拟仿真模式平稳运行。
        q_traj_rad (np.ndarray):
            形状为 (N, 5) 的二维双精度浮点型数组，表示规划生成的 N 个时间离散时刻的 5 轴关节角度序列，
            单位为弧度 (rad)。
        duration (float):
            整段轨迹在实体机械臂上执行的总规划时长，单位为秒 (s)，默认为 3.0 秒。

    返回值:
        None (无返回值)。
    """
    if ser is None or not ser.is_open:
        print("无物理硬件连接，跳过硬件时序驱动。")
        return
```

```
    steps = len(q_traj_rad)
    dt = duration / steps
    for k in range(steps):
        q_model = np.append(q_traj_rad[k], 0.0)  # 第 6 轴夹爪保持零度
        q_control = q_model * SERVO_DIRECTION_MAP
        q_servo = q_control * SERVO_DIRECTION_MAP
        ser.write(pack_frame(q_servo))
        ser.flush()
        time.sleep(dt)



def plan_cartesian_arc(p0, p1, p2, steps=60):
    """
    根据空间三点计算圆弧几何轨迹点列（单位：mm）
    p0: 起点; p1: 中间过渡点; p2: 终点
    """
    v1 = p1 - p0
    v2 = p2 - p0
    normal = np.cross(v1, v2)
    normal_len = np.linalg.norm(normal)
    if normal_len < 1e-6:
        raise ValueError("三点共线，无法生成唯一空间圆弧！")
    n = normal / normal_len

    # 外心重心坐标解析
    a = np.linalg.norm(p1 - p2)
    b = np.linalg.norm(p0 - p2)
    c = np.linalg.norm(p0 - p1)
    alpha = a ** 2 * (b ** 2 + c ** 2 - a ** 2)
    beta  = b ** 2 * (a ** 2 + c ** 2 - b ** 2)
    gamma = c ** 2 * (a ** 2 + b ** 2 - c ** 2)
    denom = alpha + beta + gamma
    if abs(denom) < 1e-8:
        raise ValueError("几何奇异，退化三角形！")
    pc = (alpha * p0 + beta * p1 + gamma * p2) / denom
    radius = np.linalg.norm(p0 - pc)
```

```
    # 局部正交基底
    u = (p0 - pc) / radius
    v = np.cross(n, u)

    # 跨角与防折返判定
    theta1 = np.arctan2(np.dot(p1 - pc, v), np.dot(p1 - pc, u))
    theta2 = np.arctan2(np.dot(p2 - pc, v), np.dot(p2 - pc, u))
    if theta1 < 0: theta1 += 2 * np.pi
    if theta2 < 0: theta2 += 2 * np.pi

    total_angle = theta2 + 2 * np.pi if theta2 < theta1 else theta2

    # 五次多项式时间参数化
    tau = np.linspace(0, 1, steps)
    s = 10 * tau ** 3 - 15 * tau ** 4 + 6 * tau ** 5

    arc_points = np.zeros((steps, 3))
    for k in range(steps):
        th = s[k] * total_angle
        arc_points[k] = pc + radius * np.cos(th) * u + radius * np.sin(th) * v

    return arc_points


def run_movel_experiment(show_animation=True, mode="auto", motion="movel", port="COM3"):
    """任务：补全轨迹规划、仿真以及按模式执行的主控流程。"""
    pass


if __name__ == '__main__':
    parser = argparse.ArgumentParser(description="第七周实验二：笛卡尔 MoveL / MoveC")
    parser.add_argument("--mode", choices=("auto", "simulate", "hardware"), default="auto")
    parser.add_argument("--motion", choices=("movel", "movec", "both"), default="movel")
    parser.add_argument("--port", default="COM3", help="auto/hardware 模式使用，默认 COM3")
    parser.add_argument("--no-animation", action="store_true")
    args = parser.parse_args()
    if args.motion == "both":
        if args.mode != "simulate":
```

```
            parser.error("连续预览 movel 和 movec 请显式指定 --mode simulate；自动/实机一次只执行一条路径")
        for motion_name in ("movel", "movec"):
            run_movel_experiment(show_animation=not args.no_animation,
                                 mode=args.mode, motion=motion_name, port=args.port)
    else:
        run_movel_experiment(show_animation=not args.no_animation,
                             mode=args.mode, motion=args.motion, port=args.port)
```

完整示例代码:

```
# 请根据上方任务代码独立补全 MoveL 和 MoveC 实验，此处不提供完整实现。
```

1.4 实验三 附加实验 桌面推块

1.4.1 实验目的

综合运用正运动学、逆运动学及本周的关节空间与笛卡尔空间轨迹规划，将外围四个木块沿桌面移到中心区域，形成紧凑的 2×2 排列。学生自主确定工位、接触姿态和运动顺序，编写适合本组机械臂的程序，建立木块目标、实际接触点与关节指令之间的关系。

作业时保持夹爪闭合，使实际夹指接触点对准木块中心，带动木块沿桌面滑动；木块全程由桌面支撑。每块到位后，夹爪先抬离，再转移到下一块。必做任务是图 1.4-1 的就近聚合。

三幅图均为同一方向的俯视示意：蓝色方块为外围初始位置，橙色方块为内围最终位置，字母表示木块编号，黑点为目标方阵中心，红线表示各块中心的移动路线。各块逐一移动，图中多条红线不表示同时运动。

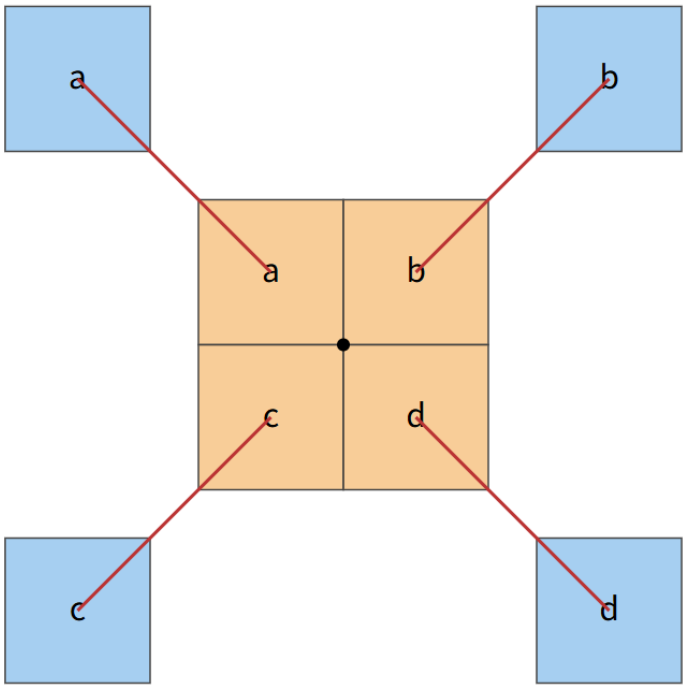


图 1.4-1 必做任务 就近聚合 上排 a b 下排 c d

将外围左上 a、右上 b、左下 c、右下 d 分别拖到中心方阵的对应位置，保持编号的相对排列。四块组成一个平面的 2×2 方阵，靠中心的四个角汇聚在黑点附近。这里是沿桌面聚合，不是上下码垛，实机允许小间隙。

全组须先完成必做任务，并获得可重复的接触、拖动和到位结果。图 1.4-2 与图 1.4-3 为选做部分，完成必做任务后可任选一种或两种继续尝试；选做结果单独记录。仿真用于检查模型与轨迹，实际成功以木块的最终摆放和完整动作过程判定。

1.4.2 实验任务与验收条件

1. 材料与初始布局：使用四个尺寸一致的立方体木块，固定标号为 a、b、c、d。标称尺寸为 30×30×30 mm，按实物测量边长 h。每次试验从相同的外围布局开始：上排 a、b，下排 c、d，四块中心位于外围正方形的四个顶点。木块边线与所选工作坐标系的 X、Y 轴平行；工作区域及桌面高度由各组标定。

2. 内围目标与任务选择：必做任务采用图 1.4-1 的排列，上排 a、b，下排 c、d。实机允许小间隙；四块不得重叠、倾倒或被抬离桌面。两项选做只改变编号与内围位置的对应关系，外围摆放和标签保持不变。读图的上下左右始终按同一俯视方向；工作区整体旋转时，起点和目标一起旋转。

选做一 同排换位

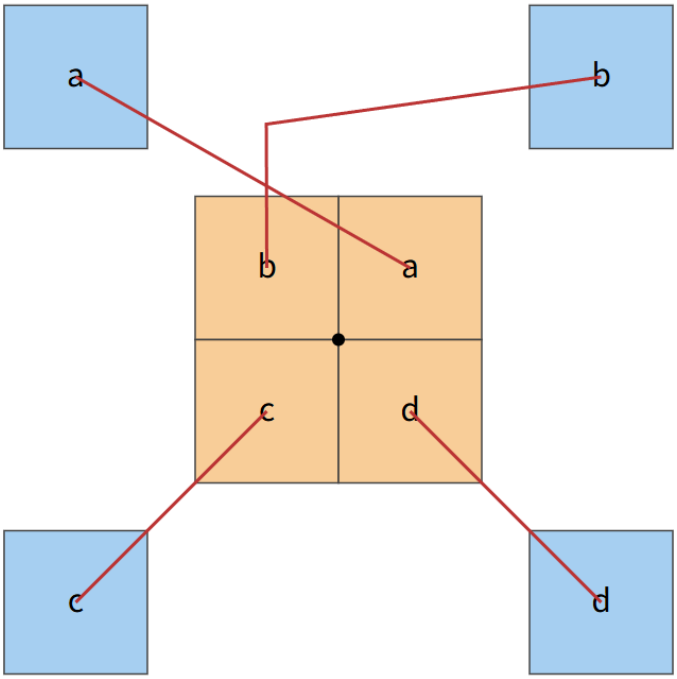


图 1.4-2 选做一 同排换位 上排 b a 下排 c d

相对必做任务，a、b 交换内围目标，c、d 保持各自目标。图中 a 移到右上目标，b 绕行至左上目标，c、d 分别移到左下、右下目标。学生应重新指定 a、b 的终点，并设计逐块顺序和必要的沿桌面绕行；不能仅改变程序循环顺序。

选做二 跨排换位

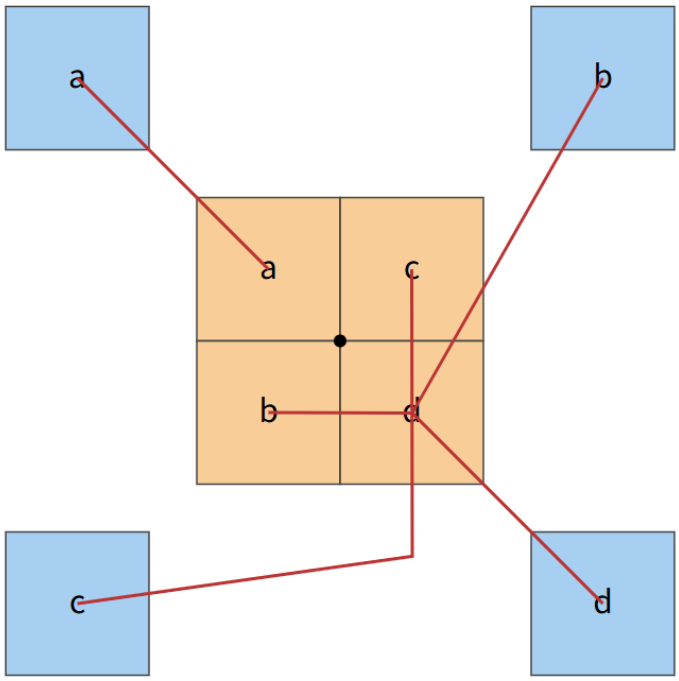


图 1.4-3 选做二 跨排换位 上排 a c 下排 b d

相对必做任务，b、c 交换内围目标，a、d 保持各自目标。b 最终到左下，c 最终到右上；图中 b、c 利用右下目标附近的区域分段转向。该区域也将放置 d，因此必须结合先后顺序和途中占用情况设计通行路线，避免拖动木块碰到已到位木块。

选做均从外围初始布局直接执行，不要求先做出必做任务方阵再交换，也不得提前互换木块或改贴标签。红线用于解释目标对应和可能的绕行方式，不是可直接下发的轨迹；实际路径、转向点和顺序由各组根据可达性、整块木块及夹指的扫掠范围规划。路线在图上交叉不代表可以相撞，执行时须分时通行并检查当时的障碍。

3. 自主设计：自行选择外围间距、中心位置、工作坐标系方向、可执行的接触姿态、夹爪闭合角、速度及动作顺序。不提供必须照搬的工位坐标、倾角或补偿数值；演示程序中的经验参数须经本组机械臂验证后使用。

4. 接触与拖动：夹爪在启动、接近、拖动、换块及结束阶段均保持闭合。实际夹指接触点应对准块顶中心，沿桌面路径带动木块；不采用夹起搬运或用夹爪持续托举木块的方式。

5. 抬离与换块：每块到位后，先执行经检查的第 2 轴单关节抬离动作，其余五轴保持原目标；确认夹爪已离开木块后，再进入高位换块姿态。该动作的模型末端沿圆弧运动，不要求固定 XY 竖直上升；抬离增量由本组规划验证，不得带动已到位木块。

6. 人工配合边界：允许在程序启动前或明确的静止阶段摆放、复位木块；运动时人员退出作业区。完整验收过程中不得用手推块、扶正或调整目标位置。一次验收失败后，应停止运动再复位，不在运动中补救。

7. 完整成功：一次连续流程完成必做任务的四块正确排列与聚合、每块接触后的抬离和最后退出，才计为必做任务完整成功。选做按各自图示排列另行验收。记录成功到位木块数、编号正确情况、相邻块最大间隙及完整流程用时；未跟随、未到位或退让时被带走的木块单独记录，不能以夹爪走完指令代替木块到位。

1.4.3 关键原理与设计提示

（1）从木块几何确定目标

设工作平面中心的局部坐标为(x_c，y_c)，木块边长为 h，名义边缘间隙为 g，支撑面高度为 z_s。四块中心在 X、Y 方向分别取正负组合，相邻中心距离为 h+g；本实验基本任务取 g=0。块顶中心的局部目标坐标可写为：

$$(x_i,\ y_i,\ z_i) = (x_c \pm (h+g)/2,\ y_c \pm (h+g)/2,\ z_s + h)$$

外围起点与内围目标使用同一个工作坐标系，再整体转换到基座坐标系。旋转工作区域时，木块的摆放方向也应随之旋转；仅对齐中心、边线方向不一致，仍可能出现顶角相碰或无法贴边。

三幅图使用相同的四个内围几何位置，区别是每个编号对应哪一位置。程序应分别表示木块编号、目标对应和执行顺序。转向或绕行时，将尚未移动和已到位木块作为障碍；检查木块和夹爪整个实体的通过空间，不能只检查末端点。

（2）区分 D-H 末端与块顶实际接触点

沿用前两实验已确认的五行 D-H 模型，第六轴仅负责夹爪开合。设闭合夹指的实际接触点相对 D-H 末端有固定偏移 Δp_tool，偏移在末端坐标系中表示，则基座坐标系中的接触点为：

$$p_c(q) = t(q) + R(q)\,\Delta p_{tool}$$

其中 t(q)和 R(q)由正运动学给出。工具坐标系中的向外不等于基座坐标系中的向上；夹爪姿态改变时，偏移在 XYZ 方向上的分量也会改变。应先测量支撑面高度、闭合后的接触位置和木块尺寸，再通过单块试验校准接触高度。

（3）在笛卡尔空间规划拖动轨迹

先生成实际接触点的路径，再逐点求解关节角并用正运动学回代。直线段可使用归一化时间 s∈[0，1]和五次插值函数：

$$w(s) = 10s^3 - 15s^4 + 6s^5$$

段内目标为 p(s)=p_start+w(s)(p_goal−p_start)。接触拖动时应保持与块顶相适应的高度，并选择同一连续逆解分支。位置逆解只约束 XYZ，不会自动保证合适的接触姿态；可选用整条路径共同可达的固定倾角，减少拖动中的翻腕变化，但固定倾角不等于完整姿态固定。

1.4.4 自主实施步骤

步骤 1 画出布局并标定。标出基座、工作平面、外围四点和中心方阵的关系，测量木块尺寸、桌面高度与闭合夹指接触点偏移，确定本组各轴实际限位。说明工作区域方向、木块摆放朝向及任务中心的选择依据。 先按图 1.4-1 标明编号对应；选做时另画目标对应及计划顺序。

步骤 2 编写可调整程序。复用或改写前两实验的模型、正解、逆解及通信函数，将起点、名义目标、逐块 Z 补偿、终点 XY 补偿、速度和抬离参数与动作流程分离。分别生成接近、压顶拖动、单独抬第 2 轴、高位换块和最终退出的轨迹。

步骤 3 离线检查完整流程。在不发送实机指令的条件下，检查所有路径的可达性、机械限位、正解误差、关节速度、构型连续性和避让。仿真观察接触点、木块排列及抬离顺序；若出现不可达、跳变或净空不足，应调整工位或姿态后重新规划。

步骤 4 验证单块拖动。先以较低速度验证一块的块顶接触、拖动到位及第 2 轴抬离。分别判断夹爪是否到达预期位置、木块是否跟随以及退出时是否被带动；根据现象检查工具偏移、Z 补偿、终点 XY 补偿、速度或接触摩擦，不同时盲改多项参数。

步骤 5 逐块校正并完成四块。保持同一桌面和摆放基准，分别校正 a、b、c、d。先固定可靠的接触参数，再微调聚合位置；每次重新检查整条路径。已到位木块应作为后续动作中的障碍，换块时先脱离并进入高位，不贴着木块横移。先完成必做任务，再在同一外围初始布局下选做图 1.4-2 或图 1.4-3；每次改变编号目标后都重新规划完整路径。

步骤 6 完成一次完整演示。启动前摆好四块并撤手，连续执行必做任务的四块排列与聚合及最终退出。记录实际编号排列、到位数量、最大间隙、用时及异常现象。重复一次检查可重复性，再在任务可靠完成的前提下优化时间；结束后返回经检查的结束姿态并关闭通信。 选做另行演示并注明所选任务。

运行方式须明确区分离线与实机。使用现有程序时，plan 只规划并导出，simulate 不连接串口，hardware 用于明确执行实机。该附加程序的 auto 模式在检测到指定串口后会直接执行实机，没有逐块回车确认；课堂调试应先明确选用离线模式，再执行经检查的实机流程。

1.4.5 提交内容与评价依据

提交本组程序、关键路径验算记录、完整演示记录和简短实验报告。演示可由教师现场验收。提交必做任务的外围与内围布局图、编号对应和执行顺序；参加选做的，另附该项路径设计和完成结果。

报告至少记录木块尺寸、任务类别、计划布局、实际编号排列、成功到位木块数、最大间隙、完整运行时间、关键点正解误差、五轴角度范围以及拖动或抬离异常。说明一次有依据的修改及修改前后结果，区分模型误差、指令误差与实机木块偏差，不将仿真数值误差写成实机精度。 必做任务与两项选做分别记录，不能以选做结果代替必做任务验收。

评价方面	本实验要求
完整任务与聚合结果	是否一次连续完成必做任务的四块正确排列、压顶拖动、聚合及退出；记录到位数量、编号、间隙和用时。选做另外记录目标排列、避让设计和实际结果。
运动学与标定	能否说明 D-H 末端、实际接触点、木块几何及工作坐标变换的关系，并提供正逆解验算和逐块补偿依据。
轨迹与退出动作	是否检查完整路径的限位、连续性、速度和实体净空；是否先仅抬第 2 轴脱离，再进行高位换块；是否带走已到位木块。
自主设计与改进	是否有可调整参数、明确流程状态与异常处理；能否解释一次失败并用修改前后实际结果说明改进效果。

评价先看任务完整性与可重复性，再比较间隙和时间。仅加快夹爪指令而导致木块打滑、漏移或退出时移位，不计为有效速度优化。